Homework #4 (HW4)

Autumn 2026 - GPU Computing

DUE: 13 November 2026, 11:59 PM

Learning Objectives

- Reinforce understanding of control divergence, coalesced memory access patterns, and other basic CUDA performance concepts.
- Get more practice using Nsight Compute (NCU) for profiling and performance analysis
- Implement the MMULT coalesced corner turning pattern in a CUDA kernel.
    - profile to verify improved memory access pattern
- EXTRA CREDIT (4 pts): implement a matrix transpose kernel to support the corner turning kernel.

Questions

- Please turn in written (or typed) solutions to these problems. In order to receive credit you must *show your work and/or describe your reasoning/explanation* for the selected answer.

1. Assume we use a tiled matrix multiplication kernel that handles data boundary conditions only on the right edge and lower (bottom) edges. Assume that we use 32X32 tiles to process square matrices of
1,000 x 1,000. Within EACH thread block, what is the **maximal** number of warps that will have control divergence due to handling boundary conditions for loading input matrix M (review L1 slides 31-35) tiles throughout the kernel execution?

a. 32
b. 24
c. 16
d. 8

2. We are to process a 600X800 (800 pixels in the x or horizontal column direction, 600 pixels in the y or
vertical row direction) image with the PictureKernel(). That is m = 600 and n = 800.

```
__global__ void PictureKernel(float* d_Pin, float* d_Pout, int n, int m) {
      // Calculate the row # of the d_Pin and d_Pout element to process
      int Row = blockIdx.y*blockDim.y + threadIdx.y;
      // Calculate the column # of the d_Pin and d_Pout element to process
```

```
	int Col = blockIdx.x*blockDim.x + threadIdx.x;
	// each thread computes one element of d_Pout if in range
	if ((Row < m) && (Col < n)) {
		d_Pout[Row*n+Col] = 2*d_Pin[Row*n+Col];
	}
}
```

Assume that we decided to use a grid of 16X16 blocks. That is, each block is organized as a 2D 16X16 array of threads. How many warps will be generated during the execution of the kernel?

(A) 37*16
(B) 38*8*50
(C) 38*50
(D) 38*50*2

3. In Question 2, how many warps will have control divergence?

a. 37 + 50*8
b. 0
c. 38*16
d. 50

4. In Question 2, if we are to process an 800x600 picture (600 pixels in the x or horizontal direction and 800 pixels in the y or vertical direction) picture, how many warps will have control divergence?

a. 50*8
b. 37+50*8
c. 38*16
d. 0

5. In Question 2, if are to process a 799x600 picture (600 pixels in the x direction and 799 pixels in
the y direction), how many warps will have control divergence?

a. (37+50)*8
b. 50*8
c. 37+50*8
d. 0

6. Assume the following simple matrix multiplication kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
	int Row = blockIdx.y*blockDim.y+threadIdx.y;
	int Col = blockIdx.x*blockDim.x+threadIdx.x;
	if ((Row < Width) && (Col < Width)) {
		float Pvalue = 0;
		for (int k = 0; k < Width; ++k) {
			Pvalue += M[Row*Width+k] * N[k*Width+Col];}
```

```
            P[Row*Width+Col] = Pvalue;
      }
}
```

If we launch the kernel with a block size of 16X16 on a 1000x1000 matrix, how many warps will have
control divergence?

a. 1,000
b. 508
c. 1,008
d. 500

7. If a CUDA device's SM (streaming multiprocessor) can take up to 1,536 threads and up to 8 thread blocks. Which of the following block configuration would result in the largest number of threads in each SM?

a. 64 threads per block
b. 512 threads per block
c. 128 threads per block
d. 1,024 threads per block

8. Assume that we want to use each thread to calculate two output elements of a vector addition. Assume that variable i should be initialized with the index for the first element to be processed by a thread. Which of the following should be used for such initialization to allow correct, *coalesced* memory accesses to these first elements in the following statement?

```
if(i<n) C[i] = A[i] + B[i];
```

(A) int i=(blockIdx.x*blockDim.x)*2 + threadIdx;
(B) int i=(blockIdx.x*blockDim.x + threadIdx.x)*2;
(C) int i=(threadIdx.x*blockDim.x)*2 + blockIdx.x;
(D) int i=(threadIdx.x*blockDim.x + blockIdx.x)*2;

9. Continuing from Question 8, what would be the correct statement for each thread to process the second element?

(A) If (i<n) C[i+1] = A[i+1] + B[i+1];
(B) If (i+1<n) C[i+1] = A[i+1] + B[i+1];
(C) If (i+threadIdx.x < n) C[i+threadIdx.x] = A[i+threadIdx.x] + B[i+threadIdx.x];
(D) if(i+blockDim.x < n) C[i+blockDim.x] = A[i+blockDim.x] + B[i+blockDim.x];

10. Assume the following simple matrix multiplication kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
      int Row = blockIdx.y*blockDim.y+threadIdx.y;
      int Col = blockIdx.x*blockDim.x+threadIdx.x;
      if ((Row < Width) && (Col < Width)) {
```

```
            float Pvalue = 0;
            for (int k = 0; k < Width; ++k) {
                  Pvalue += M[Row*Width+k] * N[k*Width+Col];}
            P[Row*Width+Col] = Pvalue;
      }
}
```

Based on the coalesced access judging criterion in Lecture 5 (slide 35), which of the following is true?

(A) M[Row*Width+k] and N[k*Width+Col] are coalesced but P[Row*Width+Col] is not
(B) M[Row*Width+k], N[k*Width+Col] and P[Row*Width+Col] are all coalesced
(C) M[Row*Width+k] is coalesced but N[k*Width+Col] andt P[Row*Width+Col] are not
(D) M[Row*Width+k] is not coalesced but N[k*Width+Col] and P[Row*Width+Col] both are

Programming and Analysis Exercises

NOTE: Always use RELEASE (x64) build when profiling your code with Nsight Compute! Otherwise results will not be representative.

Memory Access Patterns - Profiler Analysis

For this exercise we'll use square matrices of a simple size which is conducive to analysis.

1. Use small matrices of size 256 x 256 elements.

Using NC profile the memory workload analysis performance of the MMULT kernels

2. MMULTv0: all-global memory (GMEM) - naive
3. MMULTv1: tiled/phased shared-memory (SMEM)
4. For each, profile several 2D TILE_WIDTH = thread-block dimensions.
    a. 8x8
    b. 16x16
    c. 32x32
5. Review the NCU profile report sections Memory Workload Analysis - and its Memory Chart and Memory Tables. Consider Shared Memory, L1/TEX, L2 cache, and Global Memory access statistics and analyze the transaction sizes and patterns between each memory level in terms of Bytes and Sectors transferred, Sectors/Req, and wavefronts. Also consider the L1 and L2 cache hit/miss performance metrics.
6. Describe how the thread-block size relates to the warp memory access patterns and the use and efficiency of coalesced accesses and shared memory.
    a. Consider the total # bytes read from each memory level/unit for each kernel thread, based on the kernel launch configuration and input data sizes.

b. If you see any discrepancies or unexpected numbers in the profile report, identify those and consider if the “complicating factors” discussed in L5 (slide #28) might be contributing. Why or why not?

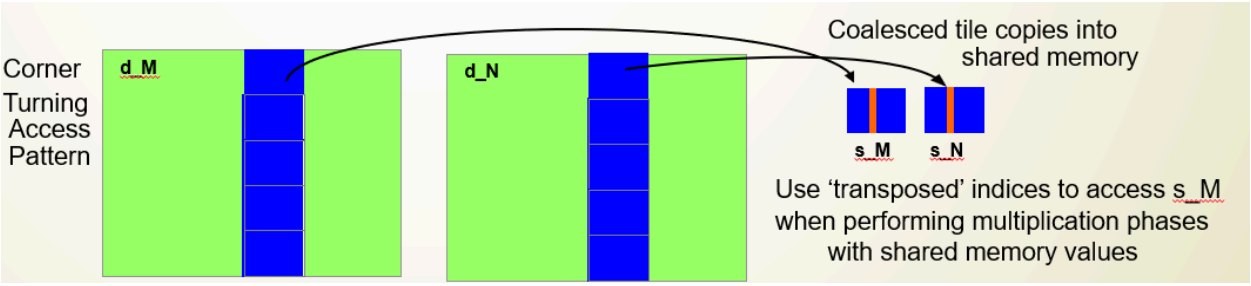
Coalesced Corner Turning Pattern: MMULTv1b Kernel

Implement an extension to the basic tiled/phased shared memory MMULT kernel which supports the corner turning algorithm, as shown in Lecture 5 slide 35 and in the image below.

As discussed in class lectures, the corner turning (transposed) access pattern allows for coalesced access to both the M and N source matrixes in GMEM.

For this HW task, It is OK to use the non-general MMULT version (which requires square matrices of a size that is an even multiple of TILE size).

In the below figure, note that matrix M must be stored in column-major layout in device memory, not row-major. As discussed in lecture 5, this will allow the accesses to M using a correct coalesced access pattern.



1. In order to setup the cornerTurning kernel input matrix data, which requires matrix M to be in column-major layout, the matrix can be transposed on the Host application prior to transferring over to the device memory.
    a. Recommended alternative is to do the EXTRA CREDIT option below. This will allow the matrix transpose to be done on the device.
2. EXTRA CREDIT (+4pts): write a CUDA kernel which performs a parallel matrix transpose on the GPU.
    a. Since global memory data has a lifetime scope of the application, the transposed matrix values will persist on the device between kernel calls, so you can first call the Transpose kernel, and then use the transposed matrix in your cornerTurning kernel, without moving the data back and forth between Host and GPU device memory in between kernel calls!

3. Take care during reading of the per-thread data from the shared memory tile during the USE phase, which must use the ‘transposed’ indices to ensure the matrix element inner-product calculation is done correctly.
    a. an alternative is to pre-transpose the elements back to row-major as they are placed into SMEM.
4. Use Host-side Windows profiling api (profileapi.h) to add high-performance timers to time the full round-trip execution of the corner-turning version of the MMULT kernel (we’ll call this **MMULTv2b**)
    a. for Windows Profiling API refer back to HW2 and L3.
5. Provide host-side verification routines to show that the same input matrices M & N will produce the same (correct) results for the following cases:
    a. the CPU sequential reference MMULT_CPU
    b. the GPU non-corner-turning versions MMULTv0/v1 (all-global and tiled/SMEM)
    c. the GPU corner-turning algorithm using transposed M matrix (**MMULTv1b)**

SUBMIT

1. Submit answers to the questions, including shown work and explained reasoning.
2. Submit any relevant CUDA host application source code
    a. .cu, .cpp and .h files only - do not submit all project & solution files and other build intermediate artifacts!
3. Submit your CUDA kernel functions
    a. may be in the same file as your host code if using a single *.cu file
4. Submit your NCU profiling reports along with a separate write-up of your analysis of the profiling results for both the Memory Access Pattern Profiler Analysis and the Coalesced Corner Turning tasks.
    a. Include your tile block-sizes info (question #4) used for the Memory Access Pattern Profiler Analysis results and discussion.